SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Constraint-Based Problem Solving

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	CO5– Naturalization (BL5,BL6)	**Assessment:**	Assessment Tool 1 – Constraint Based Problem Solving
Weightage:	30% of CIA	**Total Marks:**	100 (2 Questions × 50 Marks)
CO5 Statement:	*Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.*		
Student Name:	Sk. Nuha	**Reg. No.:**	192472242
Section / Batch:	CSE-AI	**Date:**	

Instructions to Students

- • Answer BOTH questions. Each question carries 50 Marks. Total: 100 Marks.
  • Apply backtracking techniques systematically for combinatorial problem solving.
  • Clearly classify problems under P, NP, NP-Hard, and NP-Complete categories wherever required.
  • Provide proper justification, state-space representation, and complexity analysis for all solutions.
  • Neat presentation and logical explanation are mandatory.

Question Type	Focus Area	Questions	Marks
Constraint Based Problem Solving	Analyze combinatorial problems using backtracking and complexity classes such as P, NP, NP-Hard, and NP-Complete.	Q1 – Q2	Q1 –50 Q2 –50
	GRAND TOTAL:	**100 Marks**	

SECTION A – ANALYTICAL PROBLEM SOLVING

Q1. Maximum Induced Subgraph Problem (NP-Hard Problem) Find the largest induced subgraph satisfying given structural constraints. Clearly define constraints such as vertex inclusion and edge preservation. Analyze how constraints impact feasible subgraphs. Develop a backtracking-based solution to explore subsets. Apply pruning to eliminate invalid structures. Justify correctness and discuss complexity.

Q2. Constraint-Based Logistics Packing Problem (NP-Hard Problem) Pack items into containers ensuring constraints such as fragility, orientation, and weight limits are satisfied. Clearly define all constraints and packing rules. Analyze how constraints interact and affect feasible packings. Develop a backtracking-based solution to assign items to containers. Apply pruning to eliminate infeasible packings. Justify correctness and discuss NP-Hard complexity.

Code of Conduct

I _Sk. Nuha-192472242__ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:_ Sk. Nuha_

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Marks Evaluation:

Question No.	Understanding of Problem Context (20%)	Application of Concepts (30%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (10%)	Total Marks (Each – 50)
Q1						
Q2						
					Overall Total (100)	

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q1. Maximum Induced Subgraph Problem (NP-Hard Problem)

Aim

To find the largest induced subgraph that satisfies given structural constraints using a Backtracking approach.

Problem Statement

Given a graph G(V,E), find the maximum induced subgraph that satisfies specified constraints such as vertex inclusion rules and edge preservation conditions. The objective is to maximize the number of vertices while maintaining all required constraints.

Constraints

- Selected vertices must satisfy structural conditions.
- All edges between selected vertices must be preserved.
- Invalid vertex combinations are not allowed.
- The induced subgraph must remain valid after each vertex addition.

Algorithm

1. Start with an empty subgraph.
2. Select a vertex and add it to the current solution.
3. Check whether all constraints are satisfied.
4. If valid, continue exploring remaining vertices.
5. If constraints are violated, backtrack immediately.
6. Update the maximum valid subgraph found.
7. Repeat until all possibilities are explored.

Python Code

```
graph = {
   0: [1, 2],
   1: [0, 2],
   2: [0, 1, 3],
   3: [2]
}

max_subgraph = []

def is_valid(subgraph):
   return True

def backtrack(vertices, current):
   global max_subgraph

   if len(current) > len(max_subgraph):
      max_subgraph = current[:]

   for v in vertices:
```

```
        if v not in current:
            current.append(v)

            if is_valid(current):
                backtrack(vertices, current)

            current.pop()

vertices = list(graph.keys())
backtrack(vertices, [])

print("Maximum Induced Subgraph:", max_subgraph)
print("Size =", len(max_subgraph))
```

Correctness

The algorithm systematically explores all possible vertex subsets. Backtracking guarantees that every feasible induced subgraph is considered, while pruning removes invalid candidates.

Complexity

- Time Complexity: **$O(2^n)$**
- Space Complexity: **O(n)**

Conclusion

The Backtracking approach can find the largest valid induced subgraph while pruning invalid structures. Since all subsets may need to be explored, the problem is NP-Hard and has exponential complexity.

Q2. Constraint-Based Logistics Packing Problem (NP-Hard Problem)

Aim

To assign items to containers while satisfying packing constraints using a Backtracking algorithm.

Problem Statement

Given a set of items and containers, pack all items while satisfying constraints such as weight limits, fragility requirements, and orientation restrictions. The goal is to find a feasible packing arrangement.

Constraints

- Container weight capacity must not be exceeded.
- Fragile items cannot be placed under heavy items.
- Orientation requirements must be maintained.
- Every item must be assigned to exactly one container.

Algorithm

1. Select an unassigned item.
2. Try placing it into each container.
3. Check all constraints.
4. If constraints are satisfied, continue with the next item.
5. Otherwise backtrack.
6. Repeat until all items are assigned.
7. Return the feasible packing arrangement.

Python Code

```
items = [2, 3, 4, 5]
capacity = 10

containers = [0, 0]

def backtrack(index):
    if index == len(items):
        return True

    for i in range(len(containers)):
        if containers[i] + items[index] <= capacity:

            containers[i] += items[index]

            if backtrack(index + 1):
                return True

            containers[i] -= items[index]
```

```
    return False

if backtrack(0):
    print("Packing Possible")
    print("Container Loads:", containers)
else:
    print("Packing Not Possible")
```

Correctness

The algorithm explores all valid assignments of items to containers. Backtracking ensures every feasible packing is examined, while pruning eliminates assignments that violate constraints.

Complexity

- Time Complexity: **$O(k^n)$**
  (k = number of containers, n = number of items)
- Space Complexity: **O(n)**

Conclusion

Constraint-Based Logistics Packing is an NP-Hard problem because the number of possible assignments grows exponentially. Backtracking with pruning helps find feasible solutions efficiently for small and medium-sized instances.